RAG-based College Support Assistant

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Computer Engineering

by

Divya Kumaar

Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai

2026-2027

Index

Abstract **1**

1 Introduction **1**

1.1 Problem statement 1

1.2 Objectives 1

1.3 Scope 2

1.4 Technologies used 2

1.5 Project repository 2

2 Application Workflow **2**

2.1 System architecture 2

2.2 Knowledge base: from PDF to chunks 3

2.3 Question answering flow 4

2.4 Fine-tuning workflow 4

2.5 Application screens 5

3 Results and Observations **6**

3.1 Functional test observations 6

3.2 Training results 7

3.3 Evaluation on held-out questions 8

3.4 Observations 8

4 Conclusion **9**

4.1 Limitations 10

4.2 Future scope 10

Abstract

Students ask a college helpdesk the same questions every year: fees, exam forms, hostel rules, certificates and scholarships. The answers already exist in handbooks, but finding them is slow and the student is often unsure which office to approach. This project builds a **College Support Assistant** that answers such questions from the college's own handbook and, for every answer, also raises a helpdesk ticket naming the responsible department, the category, the priority and the next action.

The assistant uses **Retrieval-Augmented Generation (RAG)**. A 23-page Student Helpdesk Handbook (PDF) is extracted, cleaned and split into 121 topic-level chunks with metadata. A hybrid word and character TF-IDF retriever selects the three most relevant chunks, and a similarity threshold makes the assistant refuse questions the handbook does not cover instead of inventing an answer. The language model behind the answers is swappable: a cloud API, a local Ollama model, an extractive (no-LLM) mode, or our own fine-tuned model.

For the fine-tuned model, **Qwen2.5-0.5B-Instruct** was adapted with **LoRA** and **QLoRA**, training only 4.40 M of its 494 M parameters (0.89 %). On held-out paraphrased questions, answer word-overlap F1 rose from 0.523 for the base model to 0.962 (LoRA) and 0.975 (QLoRA), and the share of exact facts reproduced rose from 51 % to 97–98 %. QLoRA's 4-bit base reduced peak training memory from 9.3 GB to 5.4 GB, which fits the 6 GB laptop GPU used. The system is delivered as a Streamlit web app (deployed on Streamlit Community Cloud) and a FastAPI REST service, with input validation, rate limiting, logging, latency measurement and 30 automated tests.

Keywords: Retrieval-Augmented Generation, LoRA, QLoRA, parameter-efficient fine-tuning, TF-IDF retrieval, Streamlit, FastAPI, helpdesk automation.

1. Introduction

1.1 Problem statement

A college helpdesk receives a large volume of repetitive queries. Each one costs staff time, and students wait for answers that are already written down somewhere. A plain chatbot built on a general-purpose LLM is not a safe replacement: the model does not know the college's fees, deadlines or contacts, and it may confidently invent them. The helpdesk also needs more than an answer. It needs to know which office should handle the request and how urgent it is.

1.2 Objectives

1. Answer student questions using only the college's official handbook, with the source page shown for every answer.
2. Refuse questions the handbook does not cover, and direct the student to a human contact.
3. Route every query by generating a ticket: department, category, priority and action.
4. Fine-tune a small open model with LoRA and QLoRA so that it answers in the college's exact wording, and compare it with the base model.

5. Deliver the system as a hosted web app and a REST API with validation, error handling, logging, response-time measurement and tests.

1.3 Scope

The knowledge base is a *Student Helpdesk Handbook* for *Northbridge College of Engineering & Technology*, a fictional sample institution. It has 15 sections (Admissions, Fees, Scholarships, Examinations, Academics, Library, Hostel, Placements, IT Services, Transport, Certificates, Student Welfare, Medical, Sports and Clubs, General Information) and 121 topics. Replacing the PDF with a real college's handbook in the same layout is enough to adapt the system.

1.4 Technologies used

`pypdf` and `reportlab` (PDF), `scikit-learn` TF-IDF (retrieval), an OpenAI-compatible cloud API (Groq), Ollama, and Hugging Face `transformers`, `peft` and `bitsandbytes` with PyTorch (LLMs and fine-tuning), Streamlit and FastAPI with Pydantic (interfaces), `pytest` (testing), GitHub and Streamlit Community Cloud (hosting).

1.5 Project repository

The source code, handbook PDF, fine-tuning scripts, results and tests are available on GitHub at `https://github.com/Dcode-7/LLM_Mini_Project`.

2. Application Workflow

2.1 System architecture

Figure 2.1 shows the complete system. The handbook is processed once at startup; each question then passes through validation, retrieval and an LLM backend before the answer and ticket are returned to the user.

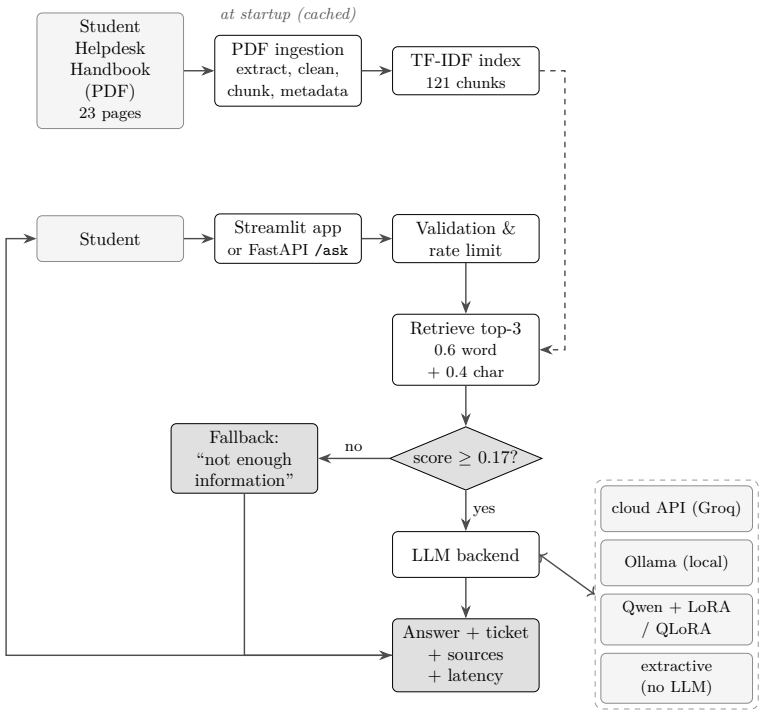


Figure 2.1: System architecture

2.2 Knowledge base: from PDF to chunks

The handbook is the only source of knowledge. Figure 2.2 shows how `pdf_ingest.py` turns it into retrievable chunks.

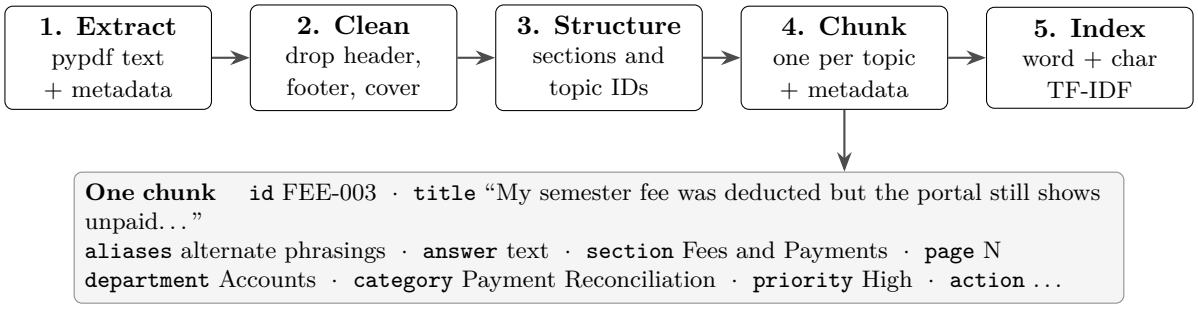


Figure 2.2: PDF ingestion pipeline and the fields stored with each chunk

On the handbook this gives 23 pages extracted, three header and footer lines removed per page, the cover and contents pages discarded, 15 sections and 121 topics detected, and 121 chunks of 190–452 characters. Answers longer than 900 characters would be split with a 150-character overlap.

Why chunk by structure and not by size? Splitting the same text every 500 characters would cut across topics, so a chunk could hold the end of one policy and the start of another, and the routing line could be separated from its answer. Chunking at topic boundaries keeps every answer whole and its ticket fields attached.

2.3 Question answering flow

1. **Validation.** The question must not be blank and must be at most 1000 characters. Each session may ask at most 30 questions per minute.
2. **Retrieval.** The question is scored against every chunk with score $= 0.6 \cdot \cos_{\text{word}} + 0.4 \cdot \cos_{\text{char}}$. Word n-grams match vocabulary; character n-grams tolerate typos and word variants ("fee", "fees").
3. **Threshold.** If the best score is below 0.17, the assistant replies that it does not have enough information and gives the front-office contact. No LLM call is made.
4. **Generation.** Otherwise the top three chunks are placed in the prompt and sent to the selected backend, which must answer only from that context.
5. **Response.** The answer is returned with a *grounded* flag, the ticket from the best chunk's metadata, the source topics with page numbers, and the response time.

2.4 Fine-tuning workflow

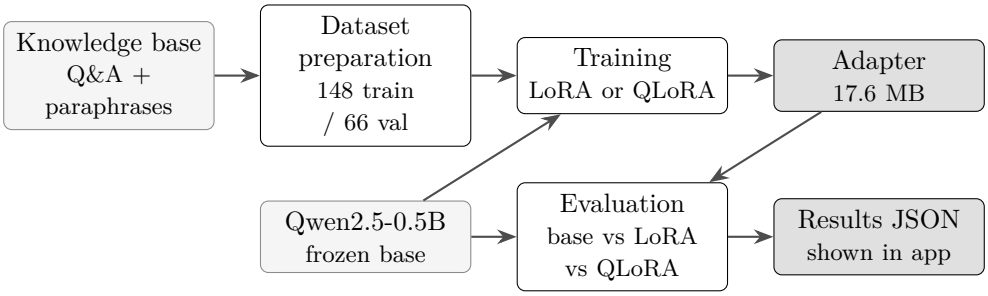


Figure 2.3: Fine-tuning workflow

Every training example uses the same retrieved-context prompt as the app. The model is trained on the canonical question and its paraphrases; one paraphrase per topic is held out for validation, together with off-topic questions that must be refused.

How LoRA works. The pretrained weight matrix W_0 is frozen. A trainable low-rank update $\Delta W = BA$ is added beside it, so the layer computes $h = W_0 x + \frac{\alpha}{r} BAx$. With $W_0 \in \mathbb{R}^{d \times k}$, $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, only $r(d + k)$ parameters are trained instead of dk. For a 4096×4096 matrix with $r = 8$ that is 256 times fewer. **QLoRA** stores the frozen base in 4-bit NF4 with double quantisation and computes in bfloat16, and uses a paged 8-bit optimiser.

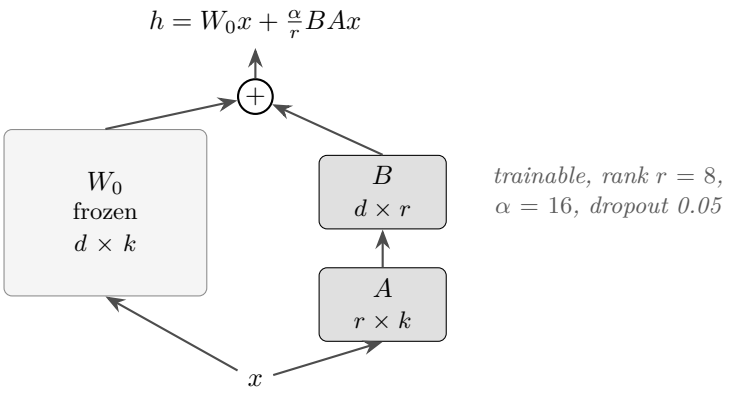


Figure 2.4: A LoRA layer: frozen base weights plus a trainable low-rank update

Configuration. Base model Qwen/Qwen2.5-0.5B-Instruct (494 M parameters); LoRA on `q,k,v,o_proj` and `gate,up,down_proj` with $r = 8$, $\alpha = 16$, dropout 0.05; 3 epochs, learning rate 2×10^{-4}, batch size 4 with gradient accumulation 2; trained on an NVIDIA RTX 3050 6 GB laptop GPU.

2.5 Application screens

The Streamlit app has three pages: the Assistant, the Knowledge base (PDF) page, and the Fine-tuning page. The FastAPI service exposes the same pipeline as a REST API.

Screenshot to add

`images/ss_answer_ticket.png`

Ask “My semester fee was deducted but the portal still shows unpaid”. Capture the answer, the ticket box (Department, Priority, Category, Action), the caption line with the response time, and the Sources expander opened.

Figure 2.5: Answer with helpdesk ticket and sources

Screenshot to add

`images/ss_kb_page.png`

Knowledge base (PDF) page, top part: Pages 23, File size, Sections 15, Chunks indexed 121, and the processing pipeline below it.

Figure 2.6: Knowledge base page: source document and pipeline

Screenshot to add

`images/ss_fastapi_docs.png`

Run `uvicorn app:app` and open `http://127.0.0.1:8000/docs`. Expand `POST /ask`, click "Try it out", send a question and capture the JSON response.

Figure 2.7: FastAPI interactive documentation

3. Results and Observations

3.1 Functional test observations

Table 3.1 lists the test cases run against the application with the extractive backend. All seven passed. The project also has 30 automated `pytest` tests covering PDF ingestion, the API endpoints and the Streamlit pages.

Table 3.1: Observation table

Test case	Expected result	Actual result	Result
Health endpoint	API reports running	`200`, status ok, backend extractive, 121 KB entries	Pass
Supported question	Grounded answer	"lost library book" → LIB-003 answer, ticket Library / Medium	Pass
Unsupported question	Fallback message	"capital of Australia" → fallback, grounded = false	Pass
Empty question	Validation error	`422` "Question must not be empty"; Streamlit shows a warning	Pass
Too-long question	Rejected	1001 characters → `422` "too long (maximum 1000 characters)"	Pass
Backend unavailable	Controlled error, no secrets	`503` backend_unavailable; reason logged server-side only	Pass
Response time	Latency recorded	`latency_ms` in every response and log line (≈4 ms extractive)	Pass

Screenshot to add

`images/ss_fallback.png`

Ask “What is the capital of Australia?” and capture the fallback answer with “not in knowledge base” in the caption line.

Figure 3.1: Out-of-scope question is refused

3.2 Training results

Table 3.2: LoRA and QLoRA training results

	LoRA	QLoRA
Frozen base stored as	bfloat16	4-bit NF4 + double quant
Trainable parameters	4.40 M (0.89 %)	4.40 M (0.89 %)
Base-weight memory	988 MB	724 MB
Peak GPU memory (training)	9.3 GB	**5.4 GB**
Training time	201 s	191 s
Final validation loss	**0.043**	0.048
Saved adapter size	17.6 MB	17.6 MB

Screenshot to add

`images/ss_training_loss.png`

Fine-tuning (LoRA / QLoRA) page, section “3 · Training loss”: the line chart with the LoRA and QLoRA curves and its legend.

Figure 3.2: Training loss per step, LoRA and QLoRA, 3 epochs

Validation loss after epochs 1, 2 and 3 was 0.045, 0.043 and 0.043 for LoRA, and 0.051, 0.048 and 0.048 for QLoRA. Both curves flatten after the second epoch, so three epochs were enough.

3.3 Evaluation on held-out questions

The validation set has 66 questions: 63 paraphrases never seen in training and 3 new off-topic questions. Every model received the same retrieved context and used greedy decoding. Figure 3.3 shows the results: answer F1 was 0.523 for the base model, 0.962 with LoRA and 0.975 with QLoRA; facts reproduced were 51 %, 97 % and 98 %; off-topic questions refused were 3/3, 3/3 and 2/3; and generation took 2.5, 8.2 and 9.7 s per answer.

Screenshot to add

`images/ss_eval_table.png`

Fine-tuning (LoRA / QLoRA) page, section "4 · Does fine-tuning help?": the table with Base, Base + LoRA and Base + QLoRA columns.

Figure 3.3: Evaluation of the base and fine-tuned models

Screenshot to add

`images/ss_finetuning_page.png`

Fine-tuning (LoRA / QLoRA) page, section "2 · LoRA vs QLoRA": the table with the LoRA and QLoRA columns (memory, rank, training time, validation loss, adapter size).

Figure 3.4: LoRA vs QLoRA comparison in the app

3.4 Observations

1. **Fine-tuning changed the behaviour, not the knowledge.** With the same context, the base model often rambled or dropped details: asked about going home from the hostel, it returned only the sentence on late entry, while both adapters gave the full leave procedure. Both adapters reproduced the college's exact answer, raising answer F1 from 0.52 to over 0.96.

2. **Under 1 % of the parameters were enough.** Only 4.40 M of 494 M parameters were trained, and the saved adapter is 17.6 MB. The full base model is still needed at inference.
3. **Weight size is not training memory.** The bf16 weights take about 1 GB, but LoRA training peaked at 9.3 GB because of activations, gradients and optimiser state, and spilled past the 6 GB VRAM into shared memory.
4. **QLoRA made training fit the GPU.** The 4-bit base brought the peak to 5.4 GB at almost the same validation loss (0.048 vs 0.043). The base is 724 MB rather than about 250 MB because embeddings and the output layer stay unquantised.
5. **QLoRA missed one refusal.** For "Recommend a good movie" it answered with the mess feedback topic. Three off-topic questions is a small sample; more refusal examples should be added to training before relying on it.
6. **LoRA did not make inference faster.** The unmerged adapter adds extra matrix multiplications, the fine-tuned answers are longer because they are complete, and the 4-bit base is dequantised on the fly.
7. **The threshold prevents invented answers.** Questions outside the handbook score below 0.17 and get the fallback message without any LLM call.
8. **Scores are helped by retrieval.** The evaluation scores are high partly because the retriever already puts the right topic in the prompt; the adapter learned to use that context faithfully, which is the intended division of work between RAG and LoRA.

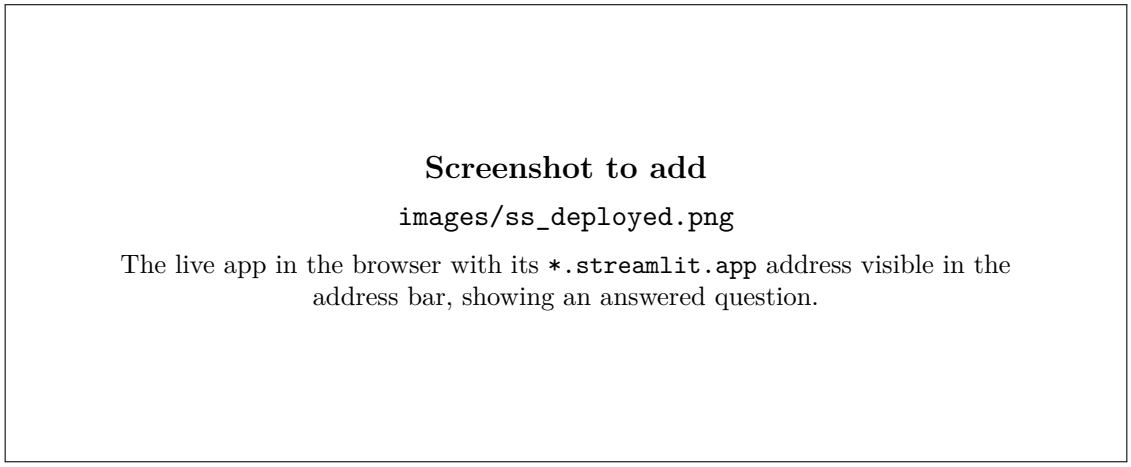


Figure 3.5: Application deployed on Streamlit Community Cloud

4. Conclusion

This project built a College Support Assistant that answers student questions from the college's own handbook and routes every query to the right office with a ticket. Processing the handbook PDF into topic-level chunks kept each answer intact with its routing details, and the similarity threshold made the assistant say "I don't know" rather than invent college policies.

Fine-tuning Qwen2.5-0.5B-Instruct with LoRA and QLoRA showed that parameter-efficient methods can adapt a model on a 6 GB laptop GPU: training under 1 % of the parameters raised

answer F1 from 0.52 to above 0.96 and fact accuracy from 51 % to 97–98 %. QLoRA matched LoRA's accuracy while cutting peak training memory from 9.3 GB to 5.4 GB. Combining the two approaches worked as intended: RAG keeps the knowledge current without retraining, and the adapter makes the model use that knowledge precisely.

The system is available as a hosted Streamlit app and a FastAPI service with validation, rate limiting, logging, latency measurement and automated tests.

4.1 Limitations

- The handbook and all its data are fictional sample data.
- TF-IDF retrieval matches words, not meaning; a question phrased very differently from the handbook may score below the threshold.
- The fine-tuned models were trained on the original 63 topics; the 58 topics added later are answered through retrieval but were not part of fine-tuning.
- The hosted app has no GPU, so it uses a cloud API or the extractive backend; the fine-tuned model runs locally.

4.2 Future scope

- Use dense embeddings (or a hybrid with TF-IDF) and a re-ranker for semantic retrieval.
- Retrain the adapters on all 121 topics with more off-topic refusal examples.
- Send tickets to a real helpdesk system and notify the responsible department.
- Add student login, multilingual support (Hindi, Marathi) and conversation memory.
- For production: authentication, HTTPS, monitoring and managed secrets.